

TataStrive Analytics: Attendance Pipeline Masterclass

Introduction & Abstract

This manual is dedicated exclusively to the **Attendance Pipeline (Cross-Day Tracking)** of TataStrive Analytics. The Attendance logic is a specialized Computer Vision engine designed solely for **Identity Verification**. It ignores student behavior and posture, focusing entirely on answering the question: *Who is this person, and have they been in this center before?*

By leveraging the **InsightFace** framework running on local edge devices, the system converts human faces into 512-dimension mathematical embeddings, matches them against a local SQLite database using cosine similarity, and streams the verified identities directly to Google BigQuery.

Phase 0: Repository Map (Attendance Components)

Logical Component	Handlers & Core Files	Location / Path
Worker Orchestration	crossday_worker.py	app/workers/
Identity Database	master_database_base.pkl / student_enrollments.db	Root / Auto-generated
Face Models	buffalo_l (onnx)	models/buffalo_l/
Data Sync Engine	bigquery_sync.py	app/
UI Configuration	crossday_tab.py	app/ui/

Part 1: The Big Picture (System Architecture)

Step 1: Understand the Pipeline

The Attendance pipeline acts as an event-driven ingestion system triggered by the **CrossDay Tab** folder listener.

- Motion Optimization:** To save CPU cycles, OpenCV background subtraction ignores static frames.
- Face Extraction:** Active frames are pushed to **InsightFace**, which detects bounds and generates facial embeddings.
- Identity Resolution:** Embeddings are mathematically compared against a known gallery of students (**G_***). If no match is found, they are registered as a "Not Found" visitor (**NF_***).
- JSON Serialization:** Entry, exit, and duration times are compiled into a localized `attendance_report.json`.
- Cloud Synchronization:** The **BigQuerySyncService** hashes the JSON, deduplicates it, and streams it to Google Cloud.

Step 2: Visualize the System Flow Sequence

```
sequenceDiagram
    participant UI as CrossDay UI (QTimer)
    participant Worker as CrossDayWorker
    participant CV as OpenCV (Motion Filter)
    participant Face as InsightFace (buffalo_l)
    participant DB as SQLite Identity DB
    participant Sync as BigQuerySyncService

    UI->>UI: Poll folder every 5 seconds
    UI->>Worker: Trigger on stable video
    Worker->>CV: Read Frame
    CV->>CV: Background Subtraction (Skip if static)
    CV->>Face: Send active frames
    Face->>Face: Generate 512-dim Embedding
    Face->>DB: Compare via Cosine Similarity
    alt Match > 0.45 Threshold
        DB-->>Worker: Map to known ID (G_102)
    else Match < 0.35 Threshold
        DB-->>DB: Save as new identity
        DB-->>Worker: Map to visitor ID (NF_04)
    end
    Worker->>Worker: Calculate Entry/Exit Duration
    Worker-->>UI: Output attendance_report.json
    UI->>Sync: Trigger Auto-Sync
    Sync->>Sync: SHA-256 Hash to prevent double-upload
```

Step 3: Trace the Payload Evolution

```
// 1. Raw Facial Extraction (Internal Memory)
{
  "bbox": [150, 200, 300, 450],
  "embedding": [0.012, -0.045, 0.991, ...] // 512 floats
}

// 2. Database Match Evaluation (Internal Memory)
{
  "best_match": "G_042",
  "cosine_similarity": 0.68,
  "confidence_score": 0.99
}

// 3. Worker Output (Local JSON Dump)
{
  "Session": { "classroom": "Lab A", "date": "2024-05-10" },
  "People": [
    {
      "id": "G_042",
      "type": "Student",
      "entry": "09:14:22",
      "exit": "10:05:00",
      "duration_sec": 3038
    }
  ]
}

// 4. BigQuery Sync Flattening (Sent to Cloud)
{
  "center_id": "CenterAlpha",
  "sync_timestamp": "2026-06-02T10:00:00.000Z",
  "report_date": "2024-05-10",
  "person_id": "G_042",
```

```
"person_type": "Student",  
"entry_time": "09:14:22",  
"exit_time": "10:05:00"  
}
```

Step 4: Review Architectural Strategies

1. **InsightFace over Standard YOLO:** While YOLO is great for finding *where* a face is, InsightFace (using the `buffalo_l` architecture) is required for Re-ID because it is specifically trained to output highly isolated facial characteristics that remain consistent across varying lighting and angles.
 2. **OpenVINO Acceleration:** Identity verification is computationally brutal on a CPU. The pipeline is built to utilize Intel OpenVINO execution providers when a dedicated GPU is absent, offering a 2x-3x speedup on Intel silicon.
-

Part 2: The Toolbelt (Prerequisites & Infrastructure)

Step 1: Standalone Executable Packaging

[!NOTE]

This application uses PyInstaller, NOT Docker. Virtualization introduces overhead and hardware-passthrough complexity on Windows machines. The Attendance pipeline relies heavily on direct memory access to the local `master_database_base.pkl` and CPU instruction sets, meaning a standalone `.exe` built via PyInstaller is the optimal deployment strategy.

Step 2: Infrastructure Sandbox

1. **Google BigQuery:** Stores the massive influx of daily attendance rows.
2. **Streaming Inserts:** The app uses `insert_rows_json` rather than batch load jobs to ensure immediate reflection on dashboards.
3. **Local Database Fallbacks:** The pipeline expects an SQLite database (`student_enrollments.db`). If absent, it gracefully degrades to legacy Pickle (`.pkl`) databases.

Step 3: AI Frameworks

1. **InsightFace:** Generates the 512-dimension vectors.
 2. **ONNX Runtime:** The backend engine running the InsightFace models, heavily reliant on `vcruntime140.dll`.
-

Part 3: The Code Deep Dive (CrossDayWorker)

Step 1: Locate the Handler

`app/workers/crossday_worker.py` acts as the execution brain for the pipeline.

Step 2: Code Walkthrough

1. Instantiated by `app.ui.crossday_tab.py`.
2. Executes pre-flight checks (`run_crossday_preflight`) to ensure ONNX and PyTorch DLLs are present.
3. Checks for `OpenVINOExecutionProvider` and falls back to `CPUExecutionProvider` if necessary.
4. Executes the `_detect_motion_segments()` block to identify timestamp bounds of active movement.
5. Loops through active motion frames, triggering `face_app.get(frame)`.
6. Executes `_note_identity_similarity()` to compare the active face against the `global_gallery`.

Step 3: Telemetry & Math (Cosine Similarity)

The mathematical engine defining identity resolution operates on these exact configured thresholds:

Threshold Variable	Default Value	Formula Application
T_STRICT_MERGE	0.45	<code>if sim > T_STRICT_MERGE:</code> definitively maps the face to the historical identity (<code>G_*</code>).
T_NEW_ID	0.35	<code>if sim < T_NEW_ID:</code> confirms the face has never been seen, forcing a new <code>NF_*</code> ID creation.
T_RATIO_MARGIN	0.10	Confirms the best match is significantly better than the second-best match (preventing collisions between twins or similar faces).

Step 4: Configure Environment Variables

Variable	Required	Description
<code>GOOGLE_APPLICATION_CREDENTIALS</code>	Yes	Path to the BigQuery service account JSON. Without this, the <code>attendance_report.json</code>

Variable	Required	Description
		n remains local and the cloud dashboards go blind.

Step 5: "What-If" Scenario (Database Failure)

What if the `student_enrollments.db` is accidentally deleted from the root directory? The worker will not crash. During boot, `_load_db()` will fail gracefully and instantiate a blank dictionary in memory. Every detected face will fall below the similarity threshold (as the DB is empty) and will be assigned a visitor ID (NF_1, NF_2). If `sync_student_roster_on_run` is enabled, a background thread will query BigQuery and reconstruct the `student_enrollments.db` dynamically.

Part 4: Operations & Troubleshooting

Step 1: Build & Deployment Commands

```
python -m venv env
.\env\Scripts\activate
pip install -r requirements_app.txt
python build_exe.py
iscc installer\TataStriveAnalytics.iss
```

Step 2: Debugging Guide (Log Analysis)

- 1. ImportError: DLL load failed:** Indicates the target machine is missing the Microsoft Visual C++ Redistributable, causing `onnxruntime` to panic. Run the bundled `vc_redist.x64.exe`.
- 2. Ghost NF IDs:** If the console shows thousands of "Not Found" IDs per minute, it means InsightFace is returning garbage embeddings due to corrupted weights. Verify the `buffalo_l` directory size.

Step 3: Common Failures Quick Reference

Symptom	Code / System Cause	Resolution
App Freezes on "Loading Models..."	ONNX Runtime is executing cold-start graph optimizations.	Normal behavior on low-end CPUs. Do not force close; allow 10-15 seconds.
BigQuery Code 403	google-auth missing credentials.json.	Place JSON inside app/Creds/credentials.json.
All Students marked as Visitors	Local Database (.pkl or .db) mismatch.	The configured student_roster_center_name does not match the actual Center ID, preventing the

Symptom	Code / System Cause	Resolution
		DB from pulling the correct embeddings.

Appendices

Appendix A: Event Field Reference (BigQuery)

Field Name	Type	Description
person_id	STRING	The canonical identity of the student (G_ . . . or NF_ . . .).
person_type	STRING	Categorization (Student vs Visitor).
entry_time	STRING	First detected appearance timestamp.
exit_time	STRING	Last detected appearance timestamp.
nf_presence	INTEGER	Session count of completely unknown identities.

Appendix B: Getting Started (For New Developers)

Because both the Attendance and Engagement pipelines run from the exact same application and `.exe`, the core environment setup is identical.

For the full setup guide (including virtual environments, API keys, and BigQuery credentials), please refer to the centralized [MASTERCLASS.md - Getting Started Guide](#).

Once your environment is configured and running (`python app/main.py`):

1. In the UI, navigate explicitly to the **CrossDay Tab**.
2. Select your video input folder and output destination.
3. The folder listener will automatically trigger `crossday_worker.py` when stable videos arrive.